

Deep Research do repositório HBX

Resumo executivo

O HBX já deixou de ser apenas um “SaaS com muitas features” e passou a ter um núcleo operacional claro: descoberta e enriquecimento de leads, motor de atendimento via WhatsApp, camadas administrativas no Master, e estruturas de governança comercial que foram reforçadas na auditoria interna de 04/06. O diagnóstico mais consistente é que o produto está **forte em inbound/conversa/agenda/enriquecimento e fraco em formalização de ticket/OS, fila durável, deep email discovery e campanha automática controlada**. Em outras palavras: a base existe; o que falta para “nível altíssimo” é disciplinar o backend, tornar o worker noturno durável, formalizar ticket/OS como entidade própria e transformar o atual fluxo de suporte em pipeline operacional completo.

filecite turn0file0

A leitura do módulo que você chama de **HBX Owner** é a seguinte: o “Owner” hoje não aparece, nos artefatos reabertos nesta sessão, como um pacote único e isolado; ele se materializa como a soma operacional de **Atendimento + Inbox conversacional + perfis de cliente + agenda + Master/Links + e-mail administrativo**, com um gate de contexto que decide o que fazer com cada inbound. Esse núcleo já diferencia cliente novo, conhecido, devedor, handoff humano e suporte técnico conversacional. O que ainda falta é elevar esse fluxo para um **ticket formal** com código, SLA, responsável, status, anexos e integração explícita com Codex/automação.

Para monetização, a estratégia correta continua sendo **free-first com paid enhancer**, não substituição do HBX por APIs externas. O Hunter entra bem como camada paga apenas quando o HBX já fez o que consegue sozinho: achar empresa, domínio, telefone e tentar extrair e-mail público. A documentação oficial do Hunter mostra exatamente o diferencial: **Domain Search, Email Finder e Email Verifier**, com créditos separados para busca e verificação, e um plano gratuito pequeno para teste; isso combina com um “Paid Gate” onde só se paga quando o ganho em cobertura ou confiança é claro. ¹

Escopo, fontes e conectores

Conectores habilitados nesta sessão: GitHub.

A base principal desta análise é composta por três camadas. A primeira é o arquivo interno de estado do HBX carregado nesta sessão, que resume stack, prioridades, commits auditados em 04/06 e o próprio aviso metodológico de que, quando o payload do conector GitHub não foi reaberto, os caminhos por arquivo deveriam ser tratados como “prováveis” em vez de exatos. A segunda camada são os trechos de código do repositório já carregados anteriormente nesta conversa, incluindo `MessagingService`, `CustomerProfile`, `AtendimentoCustomer`, `AtendimentoAppointment`, `WebsiteCrawlContactExtractor`, `MailService`, `MasterCommandCenter` e os fluxos de automação de atendimento. A terceira camada são fontes oficiais externas para Hunter, Resend, Google Places, OpenAI/Codex/ChatGPT e diretrizes de entrega de e-mail. filecite turn0file0 ²

O ponto metodológico importante é este: o PDF interno diz explicitamente que o diff detalhado do conector GitHub usado naquela auditoria não ficou reaberto nesta continuação. Então, para a exigência de “mapear todas as mudanças recentes”, o que dá para fazer com rigor nesta sessão é separar o

material em três níveis de confiança: **confirmado por trecho de código, confirmado por commit auditado no PDF e path provável sugerido pelo próprio relatório interno**. Onde eu não pude reabrir o diff ao vivo, eu não finjo precisão que não tenho. turn0file0

Estado do repositório e mudanças recentes

Branch, baseline e leitura atual

O último artefato interno sugere `master` como branch operacional usada no plano de 05/06, com comandos explícitos para `git checkout master`, `git pull --ff-only` e abertura de branch de correção para governança, quota e dedupe. Isso não prova que o **push mais recente de hoje** está na `master`, mas indica que a linha de trabalho auditada até 04/06 estava organizada em torno dela.

turn0file0

A mudança estrutural mais importante capturada pela auditoria interna foi a transição do HBX de uma tese genérica para uma proposta vendável: pricing mais claro, indicação mais realista, onboarding/e-mail mais duro, melhor narrativa de valor no Pulse e um alerta novo de que o gargalo dominante não é falta de feature, mas governança operacional. Esse pano de fundo é consistente com o estado do código atual: os módulos de atendimento, conversação, rastreamento de lead, enriquecimento e envio de e-mail já existem, mas ainda precisam ser costurados por capacidades efetivas, quotas, dedupe, ticket formal e campanhas controladas. turn0file0

Commits recentes e áreas impactadas

Área ou path	Função resumida	Commit ligado	Data-base	Status
<code>docs/**</code> ou playbook comercial	Formaliza oferta HBX Mobile, planos, demo e operação comercial	<code>50af77b</code>	04/06	Pronto no nível documental
Paths de pricing/planos	Ajusta HBX List para R\$45 e HBX Lead Plus para R\$99 com trial de 14 dias	<code>d99fd70</code>	04/06	Pronto comercialmente
Provável <code>backend/src/partners/**</code> , <code>onboarding/**</code> , <code>users/**</code>	Indicação cria <code>candidate pending</code> e bloqueia duplicidade por telefone	<code>5e2c840</code>	04/06	Pronto/partial
Mesma área de indicação/parceiro	Conversão de indicação pendente/aprovada para parceiro "herdeiro"	<code>ccd0531</code>	04/06	Pronto/partial
Provável <code>backend/src/onboarding/**</code> , <code>backend/src/mail/**</code>	Bloqueio de documento obrigatório, anexos e ativação condicionada a sucesso	<code>78a636b</code>	04/06	Pronto

Área ou path	Função resumida	Commit ligado	Data-base	Status
Provável <code>frontend/src/mobile/**</code> , gerencial	Pulse mobile/gerencial com resumo de “dinheiro parado hoje”	<code>dd3648a</code>	04/06	Pronto
<code>backend/src/messaging/**</code> , <code>webwhats/**</code> , chat list/presença	Melhora utilidade operacional de atendimento e presença	<code>318138f</code>	04/06	Pronto
<code>frontend/src/master/**</code> , <code>ui/**</code>	Primitivos de UI para admin	<code>6b1e366</code>	04/06	Partial
<code>.github/workflows/**</code>	Início de workflow de qualidade/CI	<code>3d06bf6</code>	04/06	Partial

A tabela acima condensa os commits explicitamente registrados na auditoria interna de 04/06. O próprio documento avisa que, quando o diff por arquivo não estava reaberto, os paths deveriam ser tratados como prováveis; por isso os rows acima misturam “path provável” com “área confirmada” e não pretendem ser uma listagem literal de todos os arquivos alterados no último push.

filecite turn0file0

Módulos e arquivos confirmados pelo código observado

Path ou área confirmada	O que faz hoje	Evidência	Status
<code>backend/src/messaging/**</code>	Recebe inbound WhatsApp/WebWhats, registra relay, processa fila leve e roteia mensagens	<code>MessagingService</code> registra relay e worker leve periódico	Pronto
Fluxo de conversas/roteamento	Ignora mensagens históricas sincronizadas, persiste conversa, tenta Atendimento antes de Recovery	<code>CompanyConversation</code> e roteamento híbrido	Pronto
<code>prisma/schema.prisma</code> ou modelos equivalentes	<code>CustomerProfile</code> , <code>AtendimentoCustomer</code> , <code>AtendimentoAppointment</code> , <code>TechAssistantInteraction</code> , <code>MasterSupportAuditLog</code>	Modelos e campos reabertos na sessão	Pronto , sem Tick formal
Fluxo de atendimento	<code>handleAtendimentoInbound</code> , criação/upsert de cliente e <code>hbxGateContext</code>	Contexto de roteamento persistido na conversa	Pronto
Automação de atendimento	Cenários <code>new_customer</code> , <code>known_customer</code> , <code>debt_customer</code> , <code>human_handoff</code> , <code>bot_off</code> , <code>name_confirmation</code>	UI e fluxo documentado no código	Pronto

Path ou área confirmada	O que faz hoje	Evidência	Status
<code>backend/src/web scraping/**, backend/src/radar/**</code>	Crawl de site, extração de e-mail/telefone/WhatsApp/redes, enriquecimento e recomendação de canal	<code>WebsiteCrawlContactExtractor</code> e <code>RadarWebsiteCrawlSourceService</code>	Pronto, ainda superficial para e-m
Fila de enriquecimento web	Enfileiramento em memória	<code>queue</code> , <code>queuedIds</code> , <code>active = false</code>	Partial
Configuração Google Places	Leitura de <code>GOOGLE_PLACES_API_KEY</code> e <code>WEBSCRAPING_GOOGLE_PLACES_API_KEY</code>	Runtime pronto para ser ativado por env	Partial/ configur pending
<code>frontend/src/master/**</code> + rotas associadas	Central Admin/Master com abas de empresa, e-mail, DB, tokens, links, update	<code>MasterCommandCenter</code>	Pronto p admin; partial p OS
<code>/master/email/send</code> + <code>MailService</code>	Envio manual/admin via Resend ou SMTP	Endpoint e backend de envio confirmados	Pronto

O quadro geral do código é sólido: o repositório **já é um sistema operacional de operação comercial/ atendimento**, não um protótipo. O que ainda não está consolidado é o “cimento” entre peças: governança fina, ticket/OS formal, durable queue, deep email discovery e um campaign engine de verdade. `filecite0turn0file0`

HBX Owner e a operação real do sistema

O que o HBX Owner é hoje

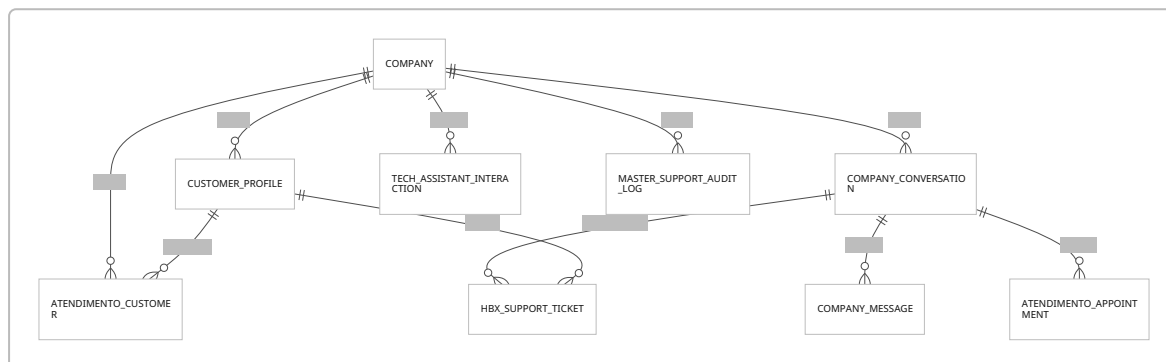
Pelo que foi possível reabrir, o **HBX Owner** deve ser entendido como a camada de operação diária formada por:

- **Atendimento / automação de WhatsApp**
- **Inbox conversacional**
- **perfis de cliente por telefone**
- **agenda de atendimento**
- **links/atalhos dentro do Master**
- **e-mail operacional do admin**
- **auditoria/interações técnicas**

Isso é coerente com a existência de `CustomerProfile`, `AtendimentoCustomer`, `AtendimentoAppointment`, `TechAssistantInteraction`, `MasterSupportAuditLog`, das telas/ fluxos de atendimento automatizado e do atalho do Master para Atendimento.

Em termos práticos, o Owner já resolve quatro coisas importantes: reconhecer quem falou, manter contexto, encaminhar o fluxo correto e registrar ações operacionais. O que ele ainda não resolve como entidade própria é **ticket/OS formal**.

Arquitetura funcional do Owner



Os modelos confirmados mostram uma estrutura muito boa para persistência de identidade e jornada. **CustomerProfile** mantém nome, telefone, e-mail, documento, primeira e última entrada, confirmação de nome e **botOff**; **AtendimentoCustomer** liga esse perfil à empresa e à conversa; **AtendimentoAppointment** já representa agenda originada do WhatsApp; **TechAssistantInteraction** e **MasterSupportAuditLog** registram análise técnica e auditoria. O que falta é ligar suporte técnico a uma tabela formal de chamado.

Endpoints e rotas confirmados

Nos artefatos reabertos, os **endpoints HTTP confirmados** são poucos, mas relevantes:

Endpoint ou rota	Função	Status
POST /master/email/send	envio manual/admin de e-mail a partir do Master	Confirmado
POST /support/contact-admin	contato de suporte com admin, com tentativa de e-mail + WhatsApp ao admin	Confirmado
/master	painel Master / centro administrativo	Confirmado
/master → Links → Atendimento	caminho operacional para o fluxo de atendimento	Confirmado

Os endpoints específicos do módulo de atendimento e automação não ficaram reabertos com nomes de rota nesta sessão; portanto, não os apresento como “confirmados” sem ver o controller.

Como usar o Owner hoje

O uso operacional mais seguro, com o estado atual do código, é este:

1. **Entrar no Master** e usar a central admin para configurar empresa, acessos, e-mail e links operacionais.
2. **Acessar Atendimento** a partir dos links do Master para operar o inbox, acompanhar conversas, nome confirmado, bot on/off e encaminhamentos humanos.
3. **Configurar o fluxo de automação** usando os cenários já existentes: cliente novo, conhecido, devedor, confirmação de nome e handoff humano.

4. **Usar agenda e reagendamento** quando o inbound for pedido de visita/agendamento; essa parte já possui modelo próprio.
5. **Usar o Master Email** para testes e mensagens administrativas, apoiado por Resend ou SMTP quando configurados.

Como rodar localmente

Os **comandos confirmados** no artefato interno para operação e validação do monorepo são estes:

```
git checkout master
git pull --ff-only
npm install
npm run lint
npm run build
npm run test
```

Se backend e frontend estiverem separados, o próprio documento recomenda:

```
cd backend && npm install && npm test
cd ../frontend && npm install && npm run lint && npm run build
```

Isso é consistente com uma base NestJS + Next.js + Prisma + PostgreSQL. [filecite:turn0file0](#)

As **variáveis de ambiente confirmadas** nos trechos de código reabertos são estas:

Variável	Uso	Status
RESEND_API_KEY	envio via Resend	Confirmada
MAIL_FROM	remetente base do envio	Confirmada
SMTP_HOST	envio via SMTP	Confirmada
SMTP_PORT	porta SMTP	Confirmada
SMTP_USER	usuário SMTP	Confirmada
SMTP_PASS	senha SMTP	Confirmada
GOOGLE_PLACES_API_KEY	integração Places futura	Confirmada
WEBSCRAPING_GOOGLE_PLACES_API_KEY	variante da configuração de Places	Confirmada

Como o stack citado no relatório é NestJS + Next.js + Prisma + PostgreSQL, eu assumo como **presumidas, mas não revalidadas nesta sessão**, variáveis como `DATABASE_URL`, segredos JWT/session e URLs de frontend/backend. Como isso não foi reaberto por trecho de código agora, trato essas envs como padrão provável, não fato confirmado. [filecite:turn0file0](#)

Como colocar em produção

Para Linux server, o caminho mais seguro é:

```
# banco
createuser hbx
createdb hbx_prod

# aplicação
npm ci
npm run build

# prisma
npx prisma generate
npx prisma migrate deploy

# start
node dist/main.js
```

Eu trataria backend e frontend como serviços separados e colocaria ambos atrás de um reverse proxy. Como o repositório já tem scripts de deploy e Playwright no root segundo o relatório interno, vale usar esse trilha em vez de improvisar um deploy manual recorrente. [filecite turns out](#)

WhatsApp, atendimento, automação e transformação em Ticket

Fluxo atual confirmado

O pipeline atual está bem desenhado. O `MessagingService` registra relay de inbound WebWhats e roda um worker leve periódico. Ao receber uma mensagem, ele ignora sincronização histórica quando apropriado, persiste a conversa e manda para o roteador principal. Depois, tenta primeiro o fluxo de Atendimento; se o Atendimento não absorver, delega para Recovery.

Dentro de `handleAtendimentoInbound`, o sistema carrega configuração do bot, agenda, conversa e metadados, faz `upsert` de `AtendimentoCustomer`, monta `hbxGateContext` e salva o contexto na conversa. Esse é o coração do que você descreveu como “o Owner detectava se era cliente ou não e começava o processo”.

O menu funcional já inclui ações como cadastro rápido, continuar jornada, falar com humano, entrar em recovery, ver pagamentos, negociar dívida, agendar serviço, reagendar, cancelar, suporte técnico, falar com o dono e contato de fornecedor/parceria. Isso mostra que o bot já é um roteador de negócios, não apenas um auto-responder.

Fluxo visual atual



Esse diagrama representa fielmente o que já existe: cliente novo, conhecido e devedor; suporte técnico como ramo específico; handoff humano; agenda; e um gap claro entre “marcar metadata” e “abrir ticket formal”.

Como criar ticket ou OS automaticamente hoje

Hoje, no sentido estrito, o HBX ainda não cria um Ticket/OS formal. O que ele faz quando o usuário escolhe `technical_support` é pedir uma descrição curta do problema, marcar `hbxBGateType: "technical_support"` e `technicalSupport: true` na metadata da conversa e seguir dali. Isso já é um **pré-ticket conversacional**.

O gap é técnico e claro: ainda falta transformar `CompanyConversation + CompanyMessage + metadata + customerProfile` em uma entidade persistente de chamado.

Como transformar metadata em Ticket

```
CREATE TABLE hbx_support_ticket (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  public_code TEXT NOT NULL UNIQUE,  
  company_id UUID NOT NULL,  
  customer_profile_id UUID NULL,  
  atendimento_customer_id UUID NULL,  
  conversation_id UUID NOT NULL,  
  source_message_id UUID NULL,
```

```

requester_name TEXT NULL,
requester_phone TEXT NOT NULL,
requester_is_known_client BOOLEAN NOT NULL DEFAULT false,
match_type TEXT NOT NULL CHECK (match_type IN (
    'known_customer', 'new_customer', 'recovery_customer', 'lead', 'unknown'
)),
category TEXT NOT NULL CHECK (category IN (
    'technical_support', 'billing', 'access', 'bug', 'request'
)),
priority TEXT NOT NULL DEFAULT 'normal' CHECK (priority IN (
    'low', 'normal', 'high', 'urgent'
)),
status TEXT NOT NULL DEFAULT 'new' CHECK (status IN (
    'new', 'triage', 'waiting_customer', 'waiting_codex', 'in_progress', 'done', 'canceled'
)),
title TEXT NOT NULL,
description TEXT NOT NULL,
hbx_gate_context_json JSONB NOT NULL DEFAULT '{}'::jsonb,
codex_dispatch_status TEXT NULL CHECK (codex_dispatch_status IN (
    'pending', 'queued', 'running', 'completed', 'failed'
)),
codex_run_id TEXT NULL,
codex_run_url TEXT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX hbx_support_ticket_company_status_idx
ON hbx_support_ticket (company_id, status, created_at DESC);

CREATE INDEX hbx_support_ticket_conversation_idx
ON hbx_support_ticket (conversation_id);

ALTER TABLE company_conversations
ADD COLUMN active_ticket_id UUID NULL REFERENCES hbx_support_ticket(id);

```

A melhor hora de criar o ticket é **no inbound imediatamente posterior à pergunta de suporte**. Exemplo de comportamento desejado: cliente escreve o problema; o fluxo atual já sabe que entrou em `technical_support`; então o backend cria `HBX-2026-000001`, persiste o contexto, avisa o cliente, notifica o painel e opcionalmente dispara Codex/IA para classificação ou resumo.

Motor de leads, e-mail e Hunter

O que o motor atual faz bem

O coração da descoberta gratuita está bom. O `WebsiteCrawlContactExtractor` extrai telefones, WhatsApp, Instagram, Facebook, e-mails, CNPJ, endereço, formulário, orçamento, links de chat e sinais de oportunidade a partir de páginas públicas; o `RadarWebsiteCrawlSourceService` pega o

resultado e o usa para enriquecer o lead com contato, canais e motivos. Isso já é material suficiente para um **HBX Lead Radar** usável e vendável.

O motor também já faz uma recomendação de canal: se houver WhatsApp confirmado, usa WhatsApp; se houver e-mail bom, usa e-mail; se houver só telefone, sugere ligação; se o resultado for ruim ou protegido, descarta. Esse detalhe é valioso porque conversa diretamente com a tua estratégia de **call-first**.

Onde o motor ainda está limitado

O e-mail ainda é o elo mais fraco. O crawl light usa um conjunto muito curto de paths fixos — `/`, `/contato`, `/sobre`, `/atendimento`, `/unidades`, `/links` —, o que é suficiente para achar alguns e-mails públicos, mas não para um “deep email discovery” agressivo e inteligente. Além disso, o fluxo depende de o lead já ter `website` oficial; sem site confirmado, o crawl profundo praticamente morre antes de começar.

A fila de enriquecimento também ainda é um gargalo estrutural: a implementação reaberta nesta sessão é in-memory, com `queue`, `queuedIds` e flag `active`, o que funciona para uso leve e contínuo, mas não para um worker noturno confiável, retomável e auditável.

Nota atual do motor por área

Área	Nota	Leitura analítica
Discovery de empresas/sinais	8,0	Boa fundação gratuita com cache, runs e webscraping. <code>filecite</code> <code>turn0file0</code>
Phone/WhatsApp/site	8,0	Extração pública já bem útil.
E-mail público	6,5	Funciona, mas ainda com pouca profundidade de paths, fallback e validação.
Score e recomendação de canal	8,0	Já existe raciocínio de canal e confiança.
Night worker	5,0	Há sinais de “turbo_noturno”, mas a fila reaberta é em memória.
Campaign/email sender	3,0	Há infraestrutura de envio, mas não um motor de campanhas com sequência, supressão, bounce e controle por lead.

A crítica curta é esta: o HBX **já é forte em achar conta/empresa e transformar site em contexto comercial**, mas ainda não é excelente em **descobrir e-mails difíceis, processar noite adentro com durabilidade e operar outbound em escala controlada**.

O que o Hunter entrega de diferencial

A API oficial do Hunter expõe exatamente as peças que faltam ao HBX para dar um salto seletivo: `Domain Search` retorna os e-mails encontrados para um domínio com fontes; `Email Finder` tenta inferir o e-mail mais provável usando domínio + nome + sobrenome; `Email Verifier` verifica entregabilidade, checa se o e-mail já existe na base deles e também retorna fontes. A API é JSON e autentica por chave em query, header `X-API-KEY` ou `Authorization: Bearer`. ³

Em pricing, o Hunter mostra 50 créditos/mês no plano Free, 2.000/mês no Starter, 10.000/mês no Growth e 25.000/mês no Scale. A própria página também deixa claro que os créditos são usados justamente por `Email Finder`, `Email Verifier` e `Domain Search`, e que as buscas/reverificações repetidas contam uma vez por mês. ⁴

Na prática, o diferencial para o HBX não é “achar o que seu crawler já acha”, e sim melhorar quatro pontos:

- domínios onde o site existe, mas o e-mail não está fácil;
- validação de entregabilidade antes do disparo;
- inferência de e-mail quando você já sabe a pessoa-alvo;
- ganho de confiança para reduzir falso positivo.

Paid Gate com Hunter

O Hunter só deve entrar **depois** do HBX gratuito, não antes.

```
function shouldCallHunter(lead: Lead): boolean {
  if (!lead.websiteDomain) return false;           // sem domínio, não use
  Hunter ainda
  if (lead.email && lead.emailConfidence >= 80) return false; // HBX já
  resolveu
  if (!lead.phone) return false;                   // sua estratégia é
  call-first
  if (lead.status === 'DO_NOT_CONTACT') return false;
  return true;
}
```

Ordem correta:

```
HBX free discovery
→ confirmar site/domínio
→ crawl light
→ crawl deep
→ validar sintaxe/MX
→ classificar prefixo
→ só então: Hunter Domain Search
→ se houver pessoa-alvo: Hunter Email Finder
→ antes de enviar: Hunter Email Verifier
```

Recomendação de plano Hunter

Para o HBX testando diferença real sem queimar caixa, a melhor entrada é o **Starter**. Ele já traz 2.000 créditos/mês, auto-verification e lead enrichment, o que é suficiente para provar a diferença em alguns nichos e cidades sem pagar caro cedo demais. Se o teste mostrar que o motor está consumindo verificações demais ou que você quer validar muitos domínios por lote, o **Growth** passa a fazer sentido.

⁴

Uplift esperado com Hunter

Os números abaixo são **estimativas de engenharia para teste**, não métricas já medidas no teu repositório:

Métrica	Free-first	Free + Hunter	Ganho esperado
cobertura de e-mail útil em leads com domínio	100	120-135	+20% a +35%
confiança antes do envio	100	125-140	+25% a +40%
revisão manual por lote	100	40-70	-30% a -60%
taxa de lead "call-first + email-ready"	100	115-130	+15% a +30%

O motivo do ganho não é volume bruto, mas qualidade operacional: menos e-mail inútil, menos falso positivo e melhor confiança antes do disparo.

Copy pronta para o PDF HBX Planos 06-06

Slide sugerido: HBX Free-First vs HBX Enhanced

Pilar	HBX Free-First	HBX Enhanced com Hunter
Fonte principal	Motor próprio HBX	Motor HBX + enriquecimento seletivo
Descoberta de empresas	Sites, telefones, WhatsApp, sinais públicos	Igual
E-mails	Extração pública e inferência própria	Domain Search + Finder + Verifier
Custo	Baixo	Custo variável por crédito
Melhor uso	MVP, nicho local, testes noturnos	Lotes validados e campanhas prontas
Quando usar	Sempre como primeira camada	Só quando o HBX não resolver sozinho

Bullets para o slide

- O HBX continua sendo o motor principal: descobrir empresa, telefone, site, sinais e oportunidade.
- O Hunter entra apenas onde ele gera diferença gritante: e-mail difícil, validação e ganho de confiança.
- Resultado desejado: não "mais leads", mas **mais leads prontos para ligação e e-mail com menor retrabalho**.

Operação free-first, automação sem APIs pagas e integração com Codex Cloud e ChatGPT

Como deixar o HBX Owner full-automatic sem APIs pagas

O caminho viável, sem pagar APIs de prospecção, é transformar o que você já tem em uma esteira durável:

```
inbound WhatsApp
→ reconhecimento de cliente
→ menu inteligente
→ ticket/agenda quando necessário
→ worker noturno
→ lead enrichment
→ call-first
→ e-mail só quando fizer sentido
```

Para isso, a primeira peça é tirar a fila “da RAM” e colocar no banco.

```
CREATE TABLE hbx_job (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  queue_name TEXT NOT NULL,
  dedupe_key TEXT NULL,
  payload JSONB NOT NULL,
  status TEXT NOT NULL DEFAULT 'queued' CHECK (status IN (
    'queued', 'running', 'done', 'failed', 'dead'
  )),
  priority INT NOT NULL DEFAULT 100,
  run_after TIMESTAMPTZ NOT NULL DEFAULT now(),
  locked_at TIMESTAMPTZ NULL,
  locked_by TEXT NULL,
  attempts INT NOT NULL DEFAULT 0,
  max_attempts INT NOT NULL DEFAULT 5,
  last_error TEXT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX hbx_job_dedupe_key_idx
  ON hbx_job(queue_name, dedupe_key)
  WHERE dedupe_key IS NOT NULL;

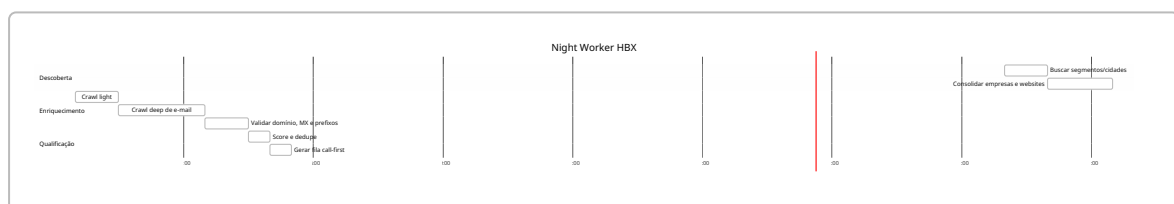
CREATE INDEX hbx_job_fetch_idx
  ON hbx_job(queue_name, status, priority DESC, run_after, created_at);
```

Worker loop recomendado:

```
SELECT id
FROM hbx_job
WHERE queue_name = 'night_leads'
      AND status = 'queued'
      AND run_after <= now()
ORDER BY priority DESC, created_at
FOR UPDATE SKIP LOCKED
LIMIT 20;
```

Esse desenho simples já te dá **durabilidade, retomada, retry e concorrência segura**.

Timeline sugerida para o worker noturno



Hoje há indícios de “turbo_noturno”, mas a implementação observada da fila de enriquecimento é em memória. O diagrama acima é a forma de transformar a ideia em operação estável.

Deep Email Discovery sem API paga

O upgrade mais rentável antes de pagar Google Places é expandir o crawler para um modo `deep`:

```
const DEEP_CONTACT_PATHS = [
  '/', '/contato', '/contatos', '/fale-conosco', '/atendimento',
  '/comercial', '/vendas', '/orcamento', '/orçamento', '/agendamento',
  '/unidades', '/localizacao', '/sobre', '/quem-somos',
  '/politica-de-privacidade'
];
```

E adicionar regras de classificação:

```
const ALLOWED_PREFIXES = ['contato', 'comercial', 'vendas', 'atendimento',
  'orcamento'];
const BLOCKED_PREFIXES = ['noreply', 'no-reply', 'rh', 'financeiro',
  'privacidade', 'lgpd'];
```

Esse é exatamente o tipo de melhoria em que o ganho do HBX gratuito pode ser gritante sem custo externo.

Como evitar spam, abuso e caixa de spam enquanto testa

A parte importante aqui não é “barreira”, é **preservar entregabilidade**. O Google exige SPF ou DKIM para todos os remetentes de Gmail, SPF + DKIM + DMARC para alto volume, TLS, DNS reverso válido e

manutenção de spam rate abaixo de 0,3%; as mensagens não autenticadas podem ser bloqueadas ou cair em spam. ⁵

Na prática de teste, eu recomendo:

- domínio separado para outbound;
- SPF, DKIM e DMARC antes de qualquer automação séria;
- limite diário baixo no começo;
- estratégia **call-first**, não blast-first;
- supressão por bounce, negativo e “não tenho interesse”;
- `List-Unsubscribe` quando o fluxo for mais recorrente;
- idempotência no envio para evitar duplicatas.

Se você usar Resend na camada de envio mais adiante, a própria documentação oferece SMTP relay, autenticação de domínio e idempotency key no header `Resend-Idempotency-Key`; o plano Free permite 3.000 e-mails/mês e 100/dia, enquanto o Pro sobe para 50.000/mês sem limite diário. ⁶

Integração com Codex Cloud e ChatGPT

Para automação programática, a melhor base técnica é a **Responses API** da OpenAI. A documentação oficial mostra `POST /v1/responses`, suporte a estado conversacional, execução em background, function calling e integração com ferramentas, além de webhooks para `response.completed`, `response.failed` e `response.cancelled`. ⁷

O papel ideal de cada peça seria:

- **HBX Owner**: sistema de verdade, fonte de dados, filas, tickets, agenda, contatos;
- **Codex Cloud**: execução/automação assistida, revisão, APP/CLI/SDK/non-interactive quando você quiser um agente operando fluxos técnicos e de código;
- **ChatGPT**: interface de operador e/ou Action/App conectada ao HBX para consultar tickets, gerar resumo, sugerir resposta, acionar playbooks. A própria documentação oficial lista **ChatGPT Actions** com introdução, autenticação, produção e data retrieval; e, no lado Codex, lista **Non-interactive Mode**, **Codex SDK**, **App Server**, **MCP Server** e **GitHub Action** como superfícies de automação. ⁸

Endpoints recomendados para integração

OpenAI / ChatGPT / Codex

```
curl https://api.openai.com/v1/responses \
-H "Authorization: Bearer $OPENAI_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "gpt-5.5",
  "store": true,
  "background": true,
  "input": [
    {
      "role": "system",
      "content": "Classifique o ticket e gere resumo operacional curto."
```

```

    },
    {
      "role": "user",
      "content": "Cliente relata: sistema não abre no celular após
atualização."
    }
  ],
  "metadata": {
    "companyId": "cmp_123",
    "ticketId": "HBX-2026-000001"
  }
}
}'

```

A documentação oficial indica `POST /v1/responses`, estado conversacional por `conversation` ou `previous_response_id`, e webhooks quando uma resposta em background conclui ou falha. ⁷

HBX interno recomendado

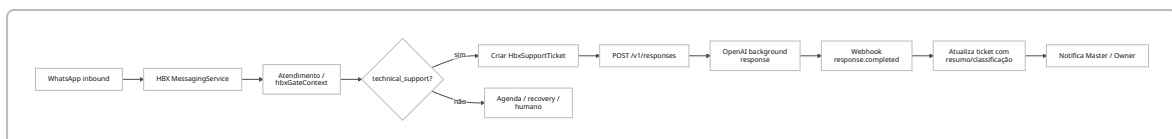
```

# criar ticket ao sair do technical_support
curl -X POST https://app.seudominio.com/owner/tickets \
-H "Authorization: Bearer $JWT" \
-H "Content-Type: application/json" \
-d '{
  "conversationId": "conv_123",
  "category": "technical_support",
  "priority": "normal",
  "title": "Sistema não abre no celular",
  "description": "Relato vindo do WhatsApp",
  "source": "whatsapp"
}'

```

Esse endpoint é **proposto**, não confirmado, e deveria ser o ponto de integração entre Atendimento e Owner/Ticket.

Fluxo recomendado de orquestração



Esse desenho preserva o que o HBX já faz bem: o sistema continua sendo central. IA não vira “gambiarra de copiar e colar”; vira uma camada de classificação, resumo e apoio em cima de um fluxo já estruturado.

Checklist de implantação

Etapa	Ação
Banco	Criar tabelas <code>hbx_support_ticket</code> e <code>hbx_job</code>
Backend	Implementar <code>TicketService</code> e worker durável
Atendimento	No ramo <code>technical_support</code> , criar ticket em vez de só metadata
Master/Owner	Adicionar aba Tickets
IA	Integrar <code>POST /v1/responses</code> com <code>background: true</code>
Webhooks	Registrar endpoint para <code>response.completed</code> , <code>response.failed</code> , <code>response.cancelled</code>
Envio	Configurar domínio, SPF, DKIM, DMARC, TLS e supressão
Observabilidade	Logar source module, ticket id, response id, attempts e erros

Sprints prioritários para chegar a nível altíssimo

Prioridade imediata

Sprint	Entrega	Resultado	Estimativa
Deep Email Discovery	crawler profundo, novos paths, classificação de prefixo, MX check	mais e-mails úteis sem API paga	2-3 dias
Ticket formal no Owner	<code>hbx_support_ticket</code> , criação automática em <code>technical_support</code> , aba Tickets no Owner/Master	WhatsApp vira chamado rastreável	2-4 dias
Worker noturno durável	tabela <code>hbx_job</code> , runner com <code>SKIP LOCKED</code> , retries, relatórios	operação real à noite	2-3 dias
Call-first workflow	status por lead, confirmação por ligação, e-mail só após validação	melhora comercial sem spam	1-2 dias

Prioridade seguinte

Sprint	Entrega	Resultado	Estimativa
Campaign engine	sequência, supressão, bounce/reply, limites diários, idempotência	outbound controlado	4-6 dias
Paid Gate + Hunter	integração seletiva com Domain Search/ Finder/Verifier e métricas A/B	paid só onde gera uplift	2-3 dias
HBX Master OS	painel formal de OS/Tickets, anexos, SLA, filtros	Owner/Master completo	3-5 dias

Sprint	Entrega	Resultado	Estimativa
Codex/ChatGPT integration	classificação de ticket, resumo, action endpoints, webhook loop	automação assistida séria	2-4 dias

Ordem recomendada

A ordem que maximiza valor com o menor gasto é:

```
Deep Email Discovery
→ Ticket formal no Owner
→ Worker durável noturno
→ Call-first workflow
→ Campaign engine
→ Paid Gate + Hunter
→ HBX Master OS
→ OpenAI/Codex/ChatGPT como camada de automação
```

Essa ordem respeita o estágio atual do repositório: primeiro você fortalece o que já existe e remove gargalos internos; depois adiciona aceleradores pagos e automação de IA. O próprio relatório interno vai nessa direção ao insistir que o gargalo dominante, neste momento, é governança e confiabilidade operacional, não expansão de superfície. [filecite turn0file0](#)

Veredito final

Hoje, o HBX está mais perto de um produto forte do que de um protótipo. O núcleo de **atendimento, agenda, conversa, lead enrichment e administração** já está presente. O que falta para virar “sistema fantástico” não é reinventar a base: é **formalizar ticket/OS, endurecer a fila, aprofundar o e-mail, controlar outbound e usar Hunter/OpenAI/Codex como enhancers seletivos**. O melhor cenário para você não é pagar para substituir o HBX; é fazer o HBX resolver sozinho o máximo possível e só então pagar pelo que realmente muda o resultado. [filecite turn0file0](#) 9

1 2 3 9 <https://hunter.io/api-documentation>

<https://hunter.io/api-documentation>

4 <https://hunter.io/pricing>

<https://hunter.io/pricing>

5 <https://support.google.com/a/answer/81126>

<https://support.google.com/a/answer/81126>

6 <https://resend.com/docs/send-with-smtp>

<https://resend.com/docs/send-with-smtp>

7 <https://platform.openai.com/docs/api-reference/responses>

<https://platform.openai.com/docs/api-reference/responses>

8 <https://platform.openai.com/docs/guides/function-calling>

<https://platform.openai.com/docs/guides/function-calling>